

Develop code that implements all forms of loops and iterators, including the use of for, the enhanced for loop (for-each), do, while, labels, break, and continue; and explain the values taken by loop counter variables during and after loop execution.

An enhanced "for" loop

The current for statement is quite powerful and can be used to iterate over arrays or collections. However, it is not optimized for collection iteration, simply because the iterator serves no other purpose than getting elements out of the collection. The new enhanced for construct lets you iterate over collections and arrays without using iterators or index variables.

The new form of the for statement has the following syntax:

```
for ( Type Identifier : Expression ) Statement
```

Note that Expression MUST be an array or an instance of a new interface called `java.lang.Iterable`, which is meant to ease the task of enabling a type for use with the enhanced for statement. This means that the `java.util.Collection` now extends the `Iterable` interface that has the following signature:

```
package java.lang;

public interface Iterable<E> {
    /**
     * Returns an iterator over the elements in this collection.
     * There are no guarantees concerning the order in which the elements
     * are returned (unless this collection is an instance of some class
     * that provides such a guarantee).
     *
     * @return an iterator over the elements in this collection
     */
    Iterator<E> iterator();
}
```

As an example, consider the following method that uses the conventional for statement to iterate over a collection:

```
// Assume we have an instance of StringBuffer "sb"
public void oldFor(Collection c) {
    for(Iterator i = c.iterator(); i.hasNext(); ) {
        String str = (String) i.next();
        sb.append(str);
    }
}
```

With the addition of generics, the above segment of code can be rewritten using the enhanced for statement as follows:

```
// Assume we have an instance of StringBuffer "sb"
public void newFor(Collection<String> c) {
    for(String str : c) {
        sb.append(str);
    }
}
```

```
}  
}
```

The enhanced for statement can be used to iterate over an array. Consider the following segment of code for a method that calculates the sum of the elements in an array:

```
public int sumArray(int array[]) {  
    int sum = 0;  
    for(int i=0; i<array.length; i++) {  
        sum += array[i];  
    }  
    return sum;  
}
```

Using the enhanced for statement, this can be rewritten as:

```
public int sumArray(int array[]) {  
    int sum = 0;  
    for(int i : array) {  
        sum += i;  
    }  
    return sum;  
}
```

The for(:) loop is designed for iteration over *collections* and *arrays*.

Iterating over arrays:

```
// for(;;) loop  
int[] ageInfo = {12, 30, 45, 55};  
int sumAge = 0;  
for (int i = 0; i < ageInfo.length; i++) {  
    sumAge += ageInfo[i];  
}
```

```
// for(:) loop  
int[] ageInfo = {12, 30, 45, 55};  
int sumAge = 0;  
for (int element : ageInfo) {  
    sumAge += element;  
}
```

NOTE, that an array element of a primitive value CANNOT be modified in the for(:) loop.

Iterating over non-generic Collection:

```
// for(;;) loop  
Collection nameList = new ArrayList();  
nameList.add("Mikalai");  
nameList.add("Michael");  
nameList.add("Craig");  
for (Iterator it = nameList.iterator(); it.hasNext(); ) {  
    Object element = it.next();  
}
```

```
        if (element instanceof String) {
            String name = (String) element;
            ...
        }
    }
```

```
// for(:) loop
Collection nameList = new ArrayList();
nameList.add("Mikalai");
nameList.add("Michael");
nameList.add("Craig");
for (Object element : nameList) {
    if (element instanceof String) {
        String name = (String) element;
        ...
    }
}
```

Iterating over generic Collection:

```
// for(;;) loop
Collection<String> nameList = new ArrayList<String>();
nameList.add("Mikalai");
nameList.add("Michael");
nameList.add("Craig");
for (Iterator<String> it = nameList.iterator(); it.hasNext();) {
    String name = it.next();
    ...
}
```

```
// for(:) loop
Collection<String> nameList = new ArrayList<String>();
nameList.add("Mikalai");
nameList.add("Michael");
nameList.add("Craig");
for (String name : nameList) {
    ...
}
```

The syntax of the `for(:)` loop does not use an iterator for the collection.

NOTE, the `for(:)` loop DOES NOT allow elements to be removed from the collection.

```
for (Type FormalParameter : Expression) Statement
```

The `FormalParameter` must be declared in the `for(:)` loop.

The `Expression` is evaluated only once.

The type of `Expression` is `java.lang.Iterable` or an array. The `java.util.Collection` interface is retrofitted to extend the `java.lang.Iterable` interface which has the method prototype `Iterator<T> iterator()`.

When Expression is an array: the type of the array element MUST be assignable to the Type of the FormalParameter.

When Expression is an instance of a raw type `java.lang.Iterable`: the Type of the FormalParameter MUST be `Object`.

When Expression is an instance of a *parameterized* type `java.lang.Iterable<T>`: the type parameter `T` MUST be assignable to the Type of the FormalParameter.

Restrictions on the enhanced for loop:

- CANNOT be used to remove elements from a collection. The 'for-each' loop hides the iterator, so you CANNOT call `remove()`. Therefore, the 'for-each' loop is not usable for filtering:

```
// This is NOT possible with 'for-each' loop !!!
// Removes the 4-letter words from c
static void expurgate(Collection<String> c) {
    for (Iterator<String> i = c.iterator(); i.hasNext(); ) {
        if (i.next().length() == 4) {
            i.remove();
        }
    }
}
```

- CANNOT be used to modify the current slot in a collection or an array. It is not usable for loops where you need to replace elements in a list or array as you traverse it.

```
Collection<String> nameList = new ArrayList<String>();
nameList.add("Mikalai");
nameList.add("Michael");
nameList.add("Craig");
for (String name : nameList) {
    name = "Nicholas"; // WRONG ! Can not modify collection
}
System.out.println(nameList);
```

The output:

```
[Mikalai, Michael, Craig]
```

```
int[] numArray = new int[] {1,2,3,4,5};
for (int i : numArray) {
    i = 0; // WRONG ! Can not modify array !
}
System.out.println(Arrays.toString(numArray));
```

The output:

```
[1, 2, 3, 4, 5]
```

However, we CAN modify object itself:

```
Collection<StringBuffer> bufferList = new ArrayList<StringBuffer>();
bufferList.add(new StringBuffer("Mikalai"));
bufferList.add(new StringBuffer("Michael"));
bufferList.add(new StringBuffer("Craig"));
for (StringBuffer name : bufferList) {
    name.reverse(); // CORRECT ! Can modify object itself
}
System.out.println(bufferList);
```

The output:

```
[ialakiM, leahciM, giarC]
```

- CANNOT be used to simultaneously iterate over multiple collections or arrays in parallel.

[Prev](#)

Chapter 2. Flow Control

[Up](#)

[Home](#)

[Next](#)

Develop code that makes use of assertions, and distinguish appropriate from inappropriate uses of assertions.



SCDJWS 1.4
Quiz



ICAD
Study Guide



Magnet
Mocker

SCWCD 1.4
Study Guide

SCBCD 1.3
Study Guide

SCDJWS 1.4
Study Guide

IBM Test 287
Study Guide